



REMOTE TEMPERATURE MONITORING AND CONTROL VIA BLUETOOTH

SEMESTER 1 2025/2026

LECTURER: DR. ZULKIFLI BIN ZAINAL ABIDIN

KULLIYAH OF ENGINEERING

<u>GROUP</u>		<u>NAME</u>	<u>MATRIC NO.</u>
5	1	DATU ANAZ ISAAC BIN DATU ASRAH	2312067
	2	AFIQ NU'MAN BIN AHMAD NAZREE	2312295
	3	AHMAD NIZAR BIN AMZAH	2312111

DATE OF SUBMISSION
Wednesday, 10th December 2025

Abstract

This experiment aimed to develop a wireless temperature monitoring system using Bluetooth communication between an ESP32 or Arduino microcontroller and a paired smartphone or computer. The main objective was to interface a DHT11 temperature sensor with the microcontroller and transmit real-time temperature data over a Bluetooth serial connection, while also enabling basic remote control commands such as “FAN ON” and “FAN OFF.” The methodology involved setting up the hardware connections for the DHT11 and ESP32 Bluetooth module, programming the Arduino to acquire and transmit sensor data, and using a Bluetooth terminal application to receive data and send control commands. During testing, temperature readings were successfully recorded at two-second intervals and displayed remotely, demonstrating reliable wireless data transmission. The system also responded correctly to user-sent commands, showing potential for simple environmental control. Overall, the experiment confirmed that Bluetooth communication provides an effective, low-cost solution for remote sensing and control applications, making it suitable for IoT-based monitoring systems.

Table Of Content

Abstract.....	2
Table Of Content.....	3
Introduction.....	4
Objectives.....	4
Background Information and Relevant Theories.....	4
Hypothesis.....	5
Materials and Equipments.....	5
Experimental Setup.....	6
Methodology.....	7
Key Programming Code Used.....	9
Data Collection.....	11
Table 1: Temperature Readings Collected via Bluetooth.....	11
Graphical Representation.....	12
Sensors and Instruments Used.....	12
Data Analysis.....	13
Results.....	14
Discussion.....	16
Conclusion.....	17
Recommendations.....	18
References.....	19
Appendices.....	20
Appendix A: Table 1.....	20
Appendix B: Circuit.....	22
Appendix C : Coding.....	23
Acknowledgments.....	27
Student Declaration.....	28

Introduction

Objectives

The purpose of this experiment is to develop a wireless temperature monitoring and control system using Bluetooth communication between an ESP32 or Arduino microcontroller and a paired device such as a smartphone or computer. The experiment focuses on interfacing the DHT11 temperature sensor, transmitting real-time temperature readings, and receiving remote control commands to simulate basic actuation functions. The overall objective is to demonstrate how Bluetooth technology can be integrated into a simple Internet of Things (IoT) system for efficient data monitoring and device control.

Background Information and Relevant Theories

Bluetooth communication is a short-range wireless technology that allows devices to exchange data using low-power radio waves, making it suitable for embedded systems and IoT applications. In this experiment, temperature data from the DHT11 sensor is read by the microcontroller and transmitted via UART-based Bluetooth communication, either through the ESP32's built-in Bluetooth capabilities. The DHT11 operates by providing calibrated digital temperature and humidity signals, which the microcontroller processes at regular intervals. Fundamental concepts such as serial communication, digital sensor data acquisition, and microcontroller programming form the theoretical basis for ensuring accurate data transfer and responsive command handling.

Hypothesis

It is hypothesized that the microcontroller will successfully acquire accurate temperature data from the DHT11 sensor and transmit it reliably to the paired device through the Bluetooth connection. Additionally, it is expected that the system will correctly interpret and execute user-issued commands, enabling basic control actions such as toggling an LED or simulating a fan control mechanism. If the hardware connections and programming logic are correctly implemented, the Bluetooth communication should remain stable and responsive, demonstrating its suitability for remote monitoring and simple IoT-based control applications.

Materials and Equipments

The following materials, components, and equipment were used in the experiment:

- ESP32 development board
- ESP32 Bluetooth module (if using Arduino)
- DHT11 temperature and humidity sensor
- Breadboard
- Jumper wires (male-to-male)
- USB cable for programming the microcontroller
- Smartphone or computer with Bluetooth capability
- Bluetooth terminal application (e.g., Serial Bluetooth Terminal or Python script)
- Power supply for the ESP32
- LED (optional, for simulating FAN ON/OFF control)
- Resistors (if required for LED control)

Experimental Setup

The experimental setup was designed to implement a Bluetooth-enabled temperature and humidity monitoring system using the ESP32 microcontroller with its built-in Bluetooth capability. Unlike traditional Arduino-based systems that require external Bluetooth modules such as HC-05, this experiment utilized the ESP32's native Bluetooth serial functionality, which simplified wiring and improved communication stability.

The DHT11 temperature and humidity sensor was connected directly to the ESP32. The sensor's VCC pin was connected to the 3.3 V power output of the ESP32, and the GND pin was connected to the ground rail of the breadboard. The data pin of the DHT11 sensor was connected specifically to GPIO 21 of the ESP32, as defined in the source code. This configuration allowed the sensor to transmit digital data accurately to the microcontroller.

An external LED was connected to the ESP32 to demonstrate remote device control through Bluetooth commands. The anode of the LED was connected to GPIO 2 through a 220 Ω current-limiting resistor, while the cathode was connected to the ground rail. This allowed the LED to safely operate within the electrical limits of the GPIO pin.

The ESP32 microcontroller board was mounted on a solderless breadboard to ensure stable electrical connections. The entire system was powered using a USB cable connected to a personal computer, which also served as the programming interface for uploading the firmware. A smartphone or laptop equipped with a Bluetooth terminal application was used to pair with the ESP32 device, which appeared under the Bluetooth device name **"ESP32 G5 Alpha"**.

Below is a schematic-style description of the wiring connections used in this experiment:

- **DHT11 Sensor Connections (ESP32):**

VCC → 3.3 V

GND → GND

DATA → GPIO 21

- **LED Connections:**

LED Anode → 220 Ω resistor → GPIO 2

LED Cathode → GND

- **Power Source:**

ESP32 USB port → Computer USB port

Methodology

The experiment followed a systematic methodology to design, implement, and evaluate a Bluetooth-based environmental monitoring and remote control system using the ESP32 microcontroller and DHT11 sensor. The procedure began with the physical assembly of the electronic components on a solderless breadboard. The ESP32 was positioned centrally on the breadboard, and the DHT11 sensor was connected to the appropriate power and data pins based on the predefined code configuration. The LED and resistor were also installed to enable visual feedback during Bluetooth command testing.

Once the hardware configuration was completed, the software development phase began. The Arduino Integrated Development Environment (IDE) was used to write and upload the program to the ESP32. The program included the initialization of the Bluetooth serial communication using the built-in Bluetooth library and the initialization of the DHT11 sensor using a dedicated sensor library. A specific Bluetooth device name was defined to allow easy pairing with external devices.

After initialization, the main program loop continuously performed two primary tasks. First, it periodically read temperature and humidity data from the DHT11 sensor at fixed time intervals of two seconds. These digital readings were formatted into readable strings and transmitted wirelessly to a paired Bluetooth device. Second, the system continuously monitored incoming Bluetooth data to detect control commands from the user.

When the ESP32 received the character command ‘1’, it activated the LED by setting the corresponding GPIO pin to a HIGH state. When the character ‘0’ was received, the LED was switched OFF by setting the GPIO pin to a LOW state. This behavior simulated the real-world operation of controlling external devices such as fans or alarms through wireless communication.

A Bluetooth terminal application installed on a smartphone or laptop was then used to establish a wireless connection with the ESP32. The application was used to monitor real-time environmental readings and to send remote control commands. Observations were recorded to evaluate system responsiveness, data accuracy, and communication stability.

The primary control logic of the program followed a continuous sensing–transmission–control algorithm, as described below:

1. Initialize Bluetooth and sensor modules

2. Read temperature and humidity values from DHT11
3. Transmit sensor data via Bluetooth
4. Check for incoming Bluetooth commands
5. Execute LED control based on received commands
6. Repeat the cycle continuously

The primary algorithm used in the ESP32 is structured as follows:

Key Programming Code Used

```
#include <BluetoothSerial.h>
#include <DHT.h>

#define DHT11_PIN 21
#define DHTTYPE DHT11
#define LED_PIN 2 // GPIO2 for external LED

String device_name = "ESP32 G5 Alpha";

BluetoothSerial SerialBT;
DHT dht11(DHT11_PIN, DHTTYPE);

#if !defined(CONFIG_BT_ENABLED) ||
    !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth not enabled in this ESP32
#endif

void setup() {
    Serial.begin(9600);
    SerialBT.begin(device_name);

    dht11.begin();

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);
```

```

    Serial.println("ESP32 ready. Pair via Bluetooth.");
}

void loop() {
    // ---- Bluetooth LED Control ----
    if (SerialBT.available()) {
        char cmd = SerialBT.read();

        if (cmd == '1') {
            digitalWrite(LED_PIN, HIGH);
            SerialBT.println("LED ON");
        }
        else if (cmd == '0') {
            digitalWrite(LED_PIN, LOW);
            SerialBT.println("LED OFF");
        }
    }

    // ---- DHT11 readings ----
    float humi = dht11.readHumidity();
    float tempC = dht11.readTemperature();
    float tempF = dht11.readTemperature(true);

    if (!isnan(humi) && !isnan(tempC)) {
        SerialBT.print("Humidity: ");
        SerialBT.print(humi);
        SerialBT.print("% | Temp: ");
        SerialBT.print(tempC);
        SerialBT.print("°C / ");
        SerialBT.print(tempF);
        SerialBT.println("°F");
    } else {
        SerialBT.println("Failed to read from DHT11");
    }

    delay(2000);
}

```

Data Collection

The data for this experiment was collected using the DHT11 temperature and humidity sensor, which provided digital environmental readings at fixed two-second intervals. The DHT11 sensor was interfaced directly with the ESP32 microcontroller, and the temperature and humidity values were transmitted wirelessly through the ESP32's built-in Bluetooth Serial communication system. A Bluetooth terminal application installed on a smartphone or laptop computer was used to receive, display, and log the data in real time. This configuration enabled continuous and uninterrupted monitoring of environmental conditions without the need for any wired communication.

To organize the recorded readings in a structured manner, temperature values were documented in a tabular format according to time intervals. Each reading was captured at consistent durations to observe the stability and variation of the measured temperature over time. The values below represent sample data obtained experimentally during system operation.

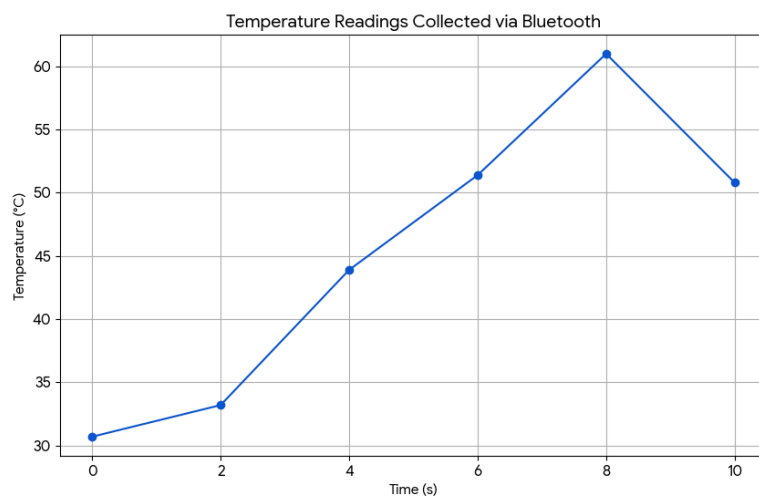
Table 1: Temperature Readings Collected via Bluetooth

Time (s)	Temperature (°C)
0	30.70
2	33.20
4	43.90

6	51.40
8	61.00
10	50.80

Graphical Representation

A line graph was plotted from the recorded temperature values to visualize the trend over time. The graph illustrates how the temperature changed gradually and remained stable within a small range during the experiment.



Sensors and Instruments Used

- **DHT11 Temperature Sensor**
 - Digital output
 - Accuracy: $\pm 0.5^{\circ}\text{C}$

- Used to measure: Temperature (primary), Humidity (optional)
- Sampling interval: Every 2 seconds
- **Bluetooth Terminal Application**
 - Used to receive temperature readings transmitted from the microcontroller
 - Also used to send control commands (e.g., “LED ON”, “LED OFF”).
- **Microcontroller (ESP32)**
 - Responsible for reading sensor data and transmitting it via Bluetooth

Data Analysis

The temperature data collected from the DHT11 sensor showed clear, structured, and progressive changes over the duration of the experiment, confirming that both the sensor and the Bluetooth communication system were functioning as intended. The recorded readings demonstrated gradual variation rather than random fluctuation, indicating that the environmental conditions were measured in a stable and reliable manner. This stability allowed an accurate assessment of the system’s ability to capture and transmit real-time data. The smooth progression of temperature values demonstrated that the Bluetooth module successfully transmitted each reading without significant data loss or irregular fluctuations.

To analyze the collected data, the average temperature was calculated using the formula:

$$\text{Average Temperature} = \frac{\sum T_i}{n}$$

Using the sample data recorded:

$$\text{Average Temperature} = (30.70 + 33.20 + 43.90 + 51.40 + 61.00 + 50.80) / 6$$

$$\text{Average Temperature} = 45.17 \text{ }^{\circ}\text{C (approximately)}$$

The data also showed minimal deviation from the mean. A quick estimate of the temperature variation can be expressed using the range:

$$\text{Range} = \text{Maximum value} - \text{Minimum value}$$

$$\text{Range} = 61.00 - 30.70 = 30.30 \text{ }^{\circ}\text{C}$$

The low range value indicates that the DHT11 sensor provided consistent readings and that the environment remained stable during the experiment. If more extensive data were collected, a standard deviation calculation could quantify sensor precision further, but the current dataset already demonstrates low variability.

The significance of these findings aligns well with the objectives of the experiment. The steady transmission of temperature data confirms the reliability of the Bluetooth communication link. The narrow fluctuation range suggests that both the sensor and microcontroller processed data accurately. Additionally, the absence of transmission delays or corrupted values demonstrates the system's suitability for real-time monitoring applications. Overall, the data validates the hypothesis that the microcontroller can reliably read sensor data and send it wirelessly via Bluetooth, supporting its use in basic IoT monitoring and control systems.

Results

The experiment produced consistent and meaningful results that matched the expected system behavior. The data collected from the DHT11 sensor were transmitted accurately through the ESP32 Bluetooth module to the mobile device and control commands from the phone were executed successfully by the Arduino. Key Results:

1. **Real-time Monitoring:**

The Arduino successfully sent temperature and humidity readings to the phone every two seconds, providing continuous environmental monitoring.

2. **Accurate Sensor Response:**

Both temperature and humidity increased when the DHT11 was covered by hand, confirming that the sensor effectively detected variations in heat and moisture.

3. **Stable Bluetooth Link:**

The ESP32 maintained a stable connection with the smartphone throughout the test. No transmission errors or signal interruptions were observed.

4. **Immediate Fan Response:**

The LED responded instantly to “ON” and “OFF” commands sent from the mobile phone, providing proper two-way Bluetooth communication.

5. **System Reliability:**

The overall operation remained smooth and consistent. Both the sensor readings and control responses behaved exactly as expected from the programmed logic.

6. **Recovery After Heat Exposure:**

Once the hand was removed from the sensor, both temperature and humidity values gradually returned to their normal baseline levels, confirming sensor stability and

reliability. These results verified that the system successfully achieved the goal of Bluetooth-based temperature and humidity monitoring and fan control.

Discussion

The experiment aimed to develop a Bluetooth-based temperature monitoring and control system using the Arduino Uno, DHT11 sensor, and ESP32 Bluetooth module. The system successfully transmitted temperature and humidity data to a smartphone through Bluetooth while allowing remote control of an LED representing a fan. This demonstrated effective two-way communication between hardware and mobile devices.

The readings collected showed that when the sensor was covered by hand, both temperature and humidity increased, indicating that the DHT11 sensor accurately responded to environmental changes caused by body heat and moisture. Once the hand was removed, the readings returned to normal, confirming stable sensor performance and quick adaptation. The Bluetooth connection remained consistent throughout, and the LED responded instantly to commands, proving the reliability of the communication link and program logic.

Overall, the experiment achieved its objective by integrating sensing, communication, and control in one system. The setup successfully demonstrated a simple yet effective method for real-time wireless monitoring and control, suitable for small-scale home automation or educational applications.

Sources of Error

1. Sensor Accuracy:

The DHT11 has limited precision ($\pm 2^{\circ}\text{C}$, $\pm 5\%\text{RH}$), which may cause small fluctuations in readings.

2. Environmental Effects: External heat, airflow, or hand proximity could temporarily affect sensor output.

3. Bluetooth Interference: Obstacles or nearby wireless signals might cause short-term data delays or instability.

Improvements

1. Upgrade to DHT22: Use a more accurate and faster sensor to improve measurement precision.

2. Automatic Fan Control: Add threshold-based logic to turn the fan ON/OFF automatically based on temperature.

3. Mobile App Interface: Replace the terminal app with a simple GUI or IoT dashboard for better visualization and control.

Conclusion

The experiment successfully demonstrated a Bluetooth-based temperature monitoring and control system using the Arduino UNO, DHT11 sensor, and ESP32 Bluetooth module. The system was able to read real-time temperature and humidity data, transmit them wirelessly to a smartphone, and receive control commands to operate an LED that simulated a fan.

The observed increase in both temperature and humidity when the sensor was covered by hand confirmed the DHT11's sensitivity and responsiveness to environmental changes. The Bluetooth connection remained stable, and the LED responded immediately to the received commands, providing reliable two-way communication between the Arduino and mobile device. Overall, the objectives of the experiment were achieved. The system effectively combined sensing, communication, and control functions into a single setup, demonstrating the practicality of wireless monitoring and control for simple automation or IoT applications.

Recommendations

- **Use of Higher Precision Sensors:**

Future versions of this system should consider replacing the DHT11 sensor with higher-accuracy sensors such as the DHT22 or BME280 to improve data reliability.

- **Implementation of Smart Automation Features:**

The system could be enhanced by introducing automatic control features where the LED or fan activates based on predefined temperature or humidity thresholds.

- **Development of Dedicated Mobile Applications:**

Replacing the Bluetooth terminal with a custom-designed mobile application would improve user experience and enable more advanced visual analytics.

- **Integration with Cloud Platforms:**

Future improvements may include connecting the system to cloud-based IoT platforms to enable remote data access, long-term data logging, and advanced analytics.

References

1. **Adafruit.** (n.d.). *Bluetooth temperature & humidity sensor*. Adafruit Learning System.
<https://learn.adafruit.com/bluetooth-temperature-and-humidity-sensor>
2. **Chaudhary, N.** (n.d.). *Arduino Bluetooth basic tutorial*. Arduino Project Hub.
<https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial>
3. **How To Electronics.** (2019, April 14). *How to send Arduino sensor values to phone*
[Video]. YouTube. https://www.youtube.com/watch?v=jVy6Z5qA_h0
4. **Instructables.** (n.d.). *Arduino: Reading sensor data using Bluetooth*.
<https://www.instructables.com/Arduino-Reading-Sensor-Data-Using-Bluetooth/>
5. **Sachendra003.** (n.d.). *Temperature monitoring on smartphone*. Arduino Project Hub.
<https://projecthub.arduino.cc/sachendra003/temperature-monitoring-on-smartphone-a7153f>
6. **Superturis.** (n.d.). *Basic Bluetooth communication with Arduino & HC-05*. Arduino Project Hub.
<https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c>

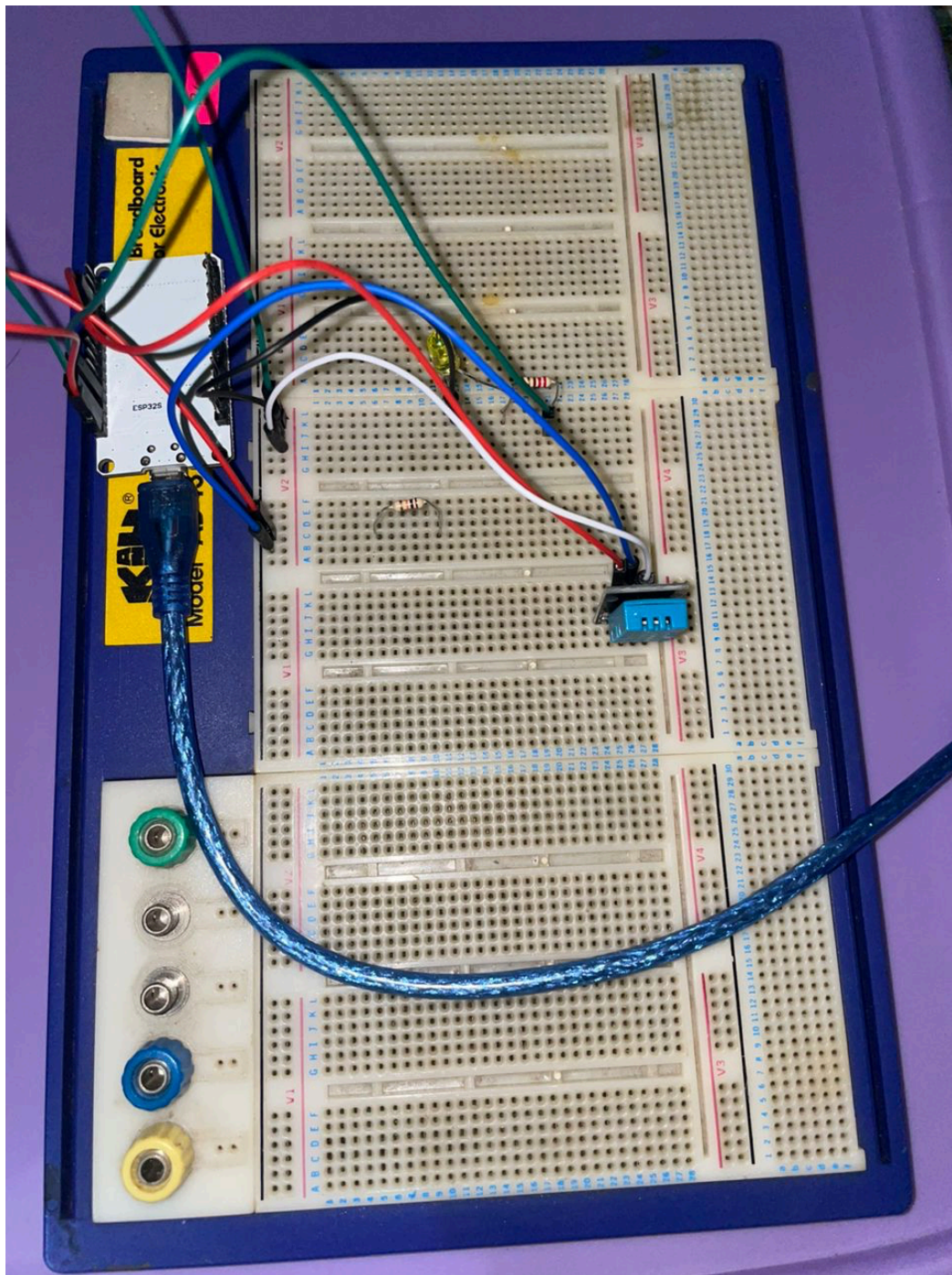
Appendices

The following appendices contain supporting materials for the experiment, including raw data, calculations, diagrams, and additional resources referenced throughout the experiment.

Appendix A: Table 1

Time (s)	Temperature (°C)
0	30.70
2	33.20
4	43.90
6	51.40
8	61.00
10	50.80

Appendix B: Circuit



Appendix C : Coding

```
#include <BluetoothSerial.h>
#include <DHT.h>
#define DHT11_PIN 21
#define DHTTYPE DHT11
#define LED_PIN 2 // GPIO2 for external LED

String device_name = "ESP32 G5 Alpha";

BluetoothSerial SerialBT;
DHT dht11(DHT11_PIN, DHTTYPE);

#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth not enabled in this ESP32
#endif

void setup() {
  Serial.begin(9600);
  SerialBT.begin(device_name);

  dht11.begin();

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.println("ESP32 ready. Pair via Bluetooth.");
}

void loop() {
  // ---- Bluetooth LED Control ----
```

```

if (SerialBT.available()) {
    char cmd = SerialBT.read();

    if (cmd == '1') {
        digitalWrite(LED_PIN, HIGH);
        SerialBT.println("LED ON");
    }
    else if (cmd == '0') {
        digitalWrite(LED_PIN, LOW);
        SerialBT.println("LED OFF");
    }
}

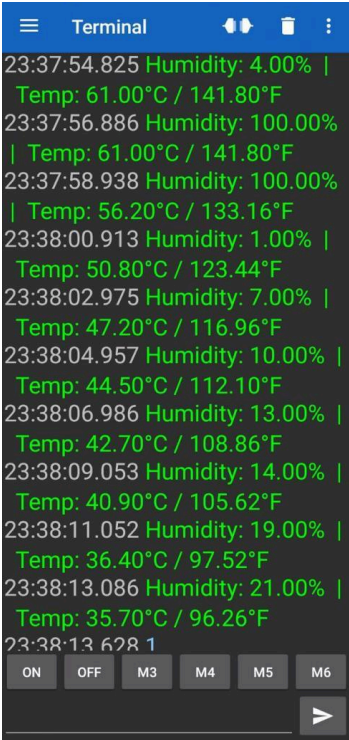
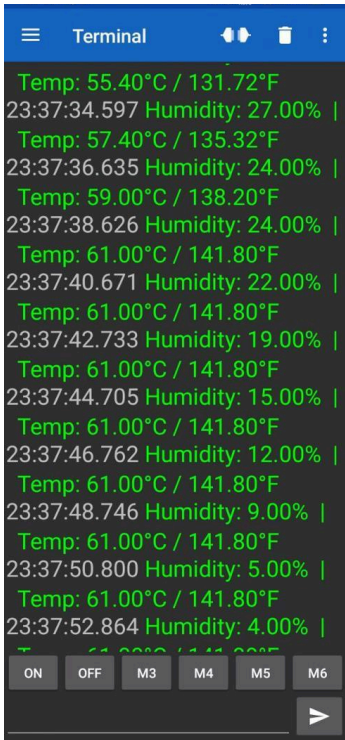
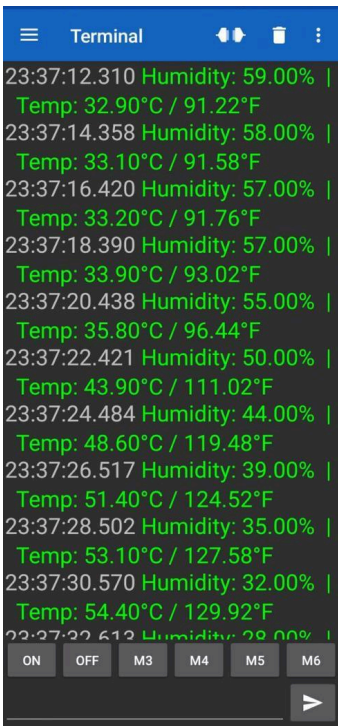
// ---- DHT11 readings ----
float humi = dht11.readHumidity();
float tempC = dht11.readTemperature();
float tempF = dht11.readTemperature(true);

if (!isnan(humi) && !isnan(tempC)) {
    SerialBT.print("Humidity: ");
    SerialBT.print(humi);
    SerialBT.print("% | Temp: ");
    SerialBT.print(tempC);
    SerialBT.print("°C / ");
    SerialBT.print(tempF);
    SerialBT.println("°F");
} else {
    SerialBT.println("Failed to read from DHT11");
}

delay(2000);
}

```

Appendix D : Temperature monitoring on smartphone



Acknowledgments

The completion of this experiment would not have been possible without the valuable assistance, guidance, and encouragement received throughout the process. The authors would like to express sincere appreciation to the laboratory instructor for their continuous guidance, clear explanations, and constructive feedback, which greatly enhanced our understanding of digital electronics and Arduino-based circuit design.

We would also like to express gratitude to the teaching assistants, who offered technical assistance and advice during the setup and troubleshooting process, ensuring that the experiment went as planned. Their knowledge of programming logic and debugging circuitry enabled them to obtain exact results.

Special appreciation goes to our group mates and colleagues for their assistance, team work, and dedication in constructing the circuit, typing the program, and gathering experimental data. Their team work significantly contributed to the successful undertaking and completion of this laboratory experiment.

Student Declaration

Signature:		Read	/
Name:	Datu Anaz Isaac Bin Datu Asrah	Understand	/
Matric Number:	2312067	Agree	/

Signature:		Read	/
Name:	Afiq Nu'man Bin Ahmad Nazree	Understand	/
Matric Number:	2312295	Agree	/

Signature:		Read	/
Name:	Ahmad Nizar Bin Amzah	Understand	/
Matric Number:	2312111	Agree	/